

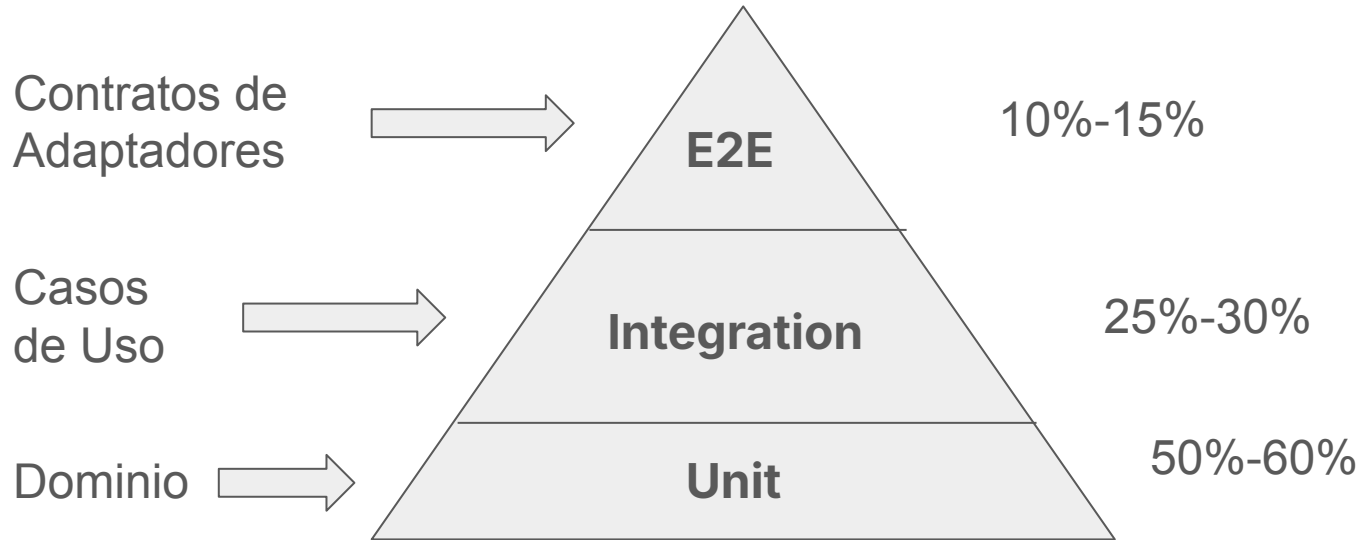
MÓDULO 2: ARQUITECTURA DE SOFTWARE

TESTING EN CLEAN ARCHITECTURE

APLICANDO CLEAN ARCHITECTURE CON TYPESCRIPT

BIG school

Pirámide de Tests en Clean Architecture



Organización

```
/tests
  /domain          # unit tests puros
  /application     # acceptance (use cases con dobles)
  /contracts       # suites compartidas (repos, event bus...)
  /e2e             # smoke/e2e con servidor real
  doubles.ts       # fakes/stubs/spies reusables
```

Nivel 1 - Dominio

```
// tests/domain/price.property.spec.ts
import { it, expect } from "vitest"
import { Price } from "@domain/value-objects/Price"

it("sumar es conmutativo (mismo currency)", () => {
  const a = Price.create(12.34, "EUR")
  const b = Price.create(5.66, "EUR")
  expect(a.add(b).amount).toBeCloseTo(b.add(a).amount, 2)
})
```

✓ Evita “mocks” en dominio.

✓ Testea **errores específicos** (p. ej. *CurrencyMismatch*).

✓ **Builders** para reducir ruido:

Nivel 1 - Dominio (Builders)

```
// tests/builders.ts
import { Price } from "@domain/value-objects/Price"
import { Quantity } from "@domain/value-objects/Quantity"
import { SKU } from "@domain/value-objects/SKU"
export const anyEUR = (n = 10) => Price.create(n, "EUR")
export const qty = (n = 1) => Quantity.create(n)
export const sku = (v = "ABC-1") => SKU.create(v)
```

Nivel 2 - Aceptación (Use Cases)

Diferencias de dobles:

- **Fake**: implementación simple en memoria (p. ej. repositorio).
- **Stub**: responde valores predeterminados (p. ej. precio actual).
- **Spy**: registra interacciones (p. ej. eventos publicados).
- **Mock**: expectativa programada (úsalo con moderación)

Nivel 2 - Aceptación (Use Cases)

```
// tests/application/addItemToOrder.events.spec.ts
import { it, expect } from "vitest"
import { InMemoryOrderRepository } from "@infra/persistence/in-memory/InMemoryOrderRep
import { StaticPricingService } from "@infra/http/StaticPricingService"
import { AddItemToOrder } from "@application/use-cases/AddItemToOrder"
import { Order } from "@domain/entities/Order"

class CapturingEventBus { public published: any[] = []; async publish(e: any[]) { this

it("publica eventos tras guardar", async () => {
  const repo = new InMemoryOrderRepository()
  await repo.save(Order.create("o-1", "c-1"))
  const pricing = new StaticPricingService({ "ABC-1": { amount: 10, currency: "EUR" } }
  const events = new CapturingEventBus()
  const uc = new AddItemToOrder(repo, pricing, events, { now: () => new Date() })

  const res = await uc.execute({ orderId: "o-1", sku: "abc-1", qty: 1, currency: "EUR"
  expect(res.ok).toBe(true)
  expect(events.published.length).toBeGreaterThan(0)
})
```

- Entrada/salida en **DTOs planos**.
- **Errores tipados** (validation/not_found/conflict).
- Sin importar **express/fastify/pg**.

Nivel 3 - Contratos de Adaptadores

Estrategia:

- **Repositorio:** guardar/leer, idempotencia, consistencia del snapshot.
- **EventBus/Outbox:** inserta eventos sin `published_at`.
- **HTTP Client** (si aplicara): timeouts, mapping a VO.

Consejos:

- Limpieza de estado entre tests (TRUNCATE).
- Igual **API** en la fábrica de la suite (factory/cleanup).

Nivel 3 - E2E/Smoke

```
// tests/e2e/flows.spec.ts
import { describe, it, expect } from "vitest"
import { buildContainer } from "@composition/container"
import { buildServer } from "@infra/http/server"

describe("E2E smoke", async () => {
  const c = buildContainer()
  const app = await buildServer(c)

  it("crea un pedido y agrega un ítem", async () => {
    const r1 = await app.inject({ method: "POST", url: "/orders", payload: { orderId:
    expect(r1.statusCode).toBe(200)

    const r2 = await app.inject({
      method: "POST", url: "/orders/o-1/items", payload: { sku: "abc-1", qty: 2, curre
    })
    expect(r2.statusCode).toBe(200)
    expect(r2.json().total.amount).toBe(20)
  })
})
```

CI con GitHub Actions

```
name: ci
on:
  push: { branches: ["main"] }
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: orders
        ports: ["5432:5432"]
        options: >-
          --health-cmd="pg_isready -U postgres -d orders"
          --health-interval=5s --health-timeout=5s --health-retries=20
    env:
      DATABASE_URL: postgresql://postgres:postgres@localhost:5432/orders
      USE_INMEMORY: "false"
      NODE_ENV: test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20.x", cache: "npm" }
      - run: npm ci
      - run: npm run db:migrate
      - run: npm test -- --run
```

IA como Copiloto

- **Diseño de suites por nivel**
“Dado este repo, lista los casos de prueba esenciales por nivel (dominio, aceptación, contratos, e2e) para minimizar riesgo y duplicidad.”
- **Generar dobles reutilizables**
“Crea fakes/stubs/spies para estos puertos {OrderRepository, PricingService, EventBus, Clock} y un archivo `tests/doubles.ts` con ejemplos de uso.”
- **Contratos robustos**
“Amplía el contrato de `OrderRepository` para incluir actualización de ítems, concurrencia simple y limpieza de estado entre pruebas.”
- **E2E estable**
“Genera un test E2E con Fastify `inject` que cubra crear pedido + agregar ítem y valide códigos de error 400/404/409.”
- **Cobertura inteligente**
“Analiza este reporte de cobertura y sugiere qué pruebas agregar en dominio y aceptación para cubrir ramas críticas sin perseguir 100%.”
- **CI con Postgres**
“Crea un workflow de GitHub Actions con servicio Postgres 16, healthcheck, migraciones y ejecución de tests con Vitest.”